

1.예제 시나리오와 프로젝트 구조

예제 시나리오와 프로젝트 구조

이번 장에서는 AI Agent 실습에 사용할 교육용 가상 제조 시나리오를 소개하고, 실습 프로젝트의 구조와 Python 실행 환경을 준비합니다.

- 박막 증착 공정 라인(LINE-07)에서 설비(EQP-EV-03)에 온도 편차 알람(ALM-TEMP-402)이 반복 발생하는 상황을 예제로 사용합니다.
- 폴더 구조와 각 파일의 역할을 이해하고, 실제 사내 데이터가 아닌 교육용 가상 데이터를 사용하는 원칙도 함께 살펴봅니다.

실습 시나리오 개요



💡 핵심 정리: Agent는 답을 바로 말하는 사람이 아니라, 자료를 모아 판단을 돕는 업무 조교입니다.

Agent는 어떻게 동작하나요?

알람 로그 확인

질문만 보고 바로 결론을 말하지 않습니다.

알람 매뉴얼 참고

관련 문서를 찾아 근거를 수집합니다.

프롬프트 생성

LLM에게 전달할 작업 지시문을 만듭니다.



이번 실습의 목표

제조 설비에서 알람이 반복 발생했을 때, AI Agent가 어떤 정보를 확인하고 어떤 순서로 답변을 준비하는지 이해하는 과정입니다.

예제 상황: 박막 증착 공정에서 온도 편차 알람이 반복되는 교육용 시나리오

LINE-07, EQP-EV-03, ALM-TEMP-402 설명

이번 실습의 핵심 시나리오에 등장하는 교육용 가상 이름들을 소개합니다.

LINE-07

교육용 가상 제조 라인입니다. 실습 데이터에서는 EDU-LINE-07처럼 표기될 수 있습니다.

EQP-EV-03

교육용 가상 설비 ID입니다. 실제 사내 설비명이 아닙니다.

ALM-TEMP-402

챔버 온도 편차와 관련된 교육용 가상 알람 코드입니다. 실제 알람 코드가 아닙니다.

  이 이름들은 실제 사내 설비명이나 실제 알람 코드가 아닙니다. 모든 이름은 Agent 구조 학습을 위한 가상 예제입니다.

사람이 알람 대응 시 확인하는 정보

사람이 알람을 대응할 때는 알람 코드 하나만 보지 않습니다. 여러 정보를 함께 확인해야 정확한 판단에 가까워집니다.



💡 핵심 정리: 제조 알람 대응은 병원 진료 기록처럼 여러 기록을 함께 읽는 과정입니다.

실습 로그에 포함된 정보 항목

항목명	설명
timestamp	알람 발생 시각
equipment_id	설비 ID
alarm_code	알람 코드
severity	심각도
measured_value	측정값
repeat_count	반복 횟수
operator_note	작업자 메모



어느 설비에서 발생했는지



언제 발생했는지



몇 번 반복되었는지



심각도는 어떤지



작업자 메모에는 무엇이 적혀 있는지

교육용 가상 데이터 사용 원칙

⚠ 이번 과정에서는 실제 사내 데이터나 실제 내부 시스템을 사용하지 않습니다.

사용하지 않는 것들

- 실제 설비명
- 실제 공정 기준
- 실제 알람 로그
- 실제 API Key
- 실제 내부 시스템명

왜 가상 데이터를 사용하나요?

보안과 안전을 위한 중요한 원칙입니다.

소방 훈련에서 실제 화재가 아니라 훈련용 상황을 사용하는 것처럼, 이번 과정도 안전한 가상 환경에서 Agent 구조를 연습합니다.

✔ 💡 핵심 정리: 실습은 실제 연결이 아니라 교육용 구조 이해가 목적입니다.

프로젝트 폴더 구조

처음 보면 파일이 많아 복잡해 보일 수 있지만, 역할별로 보면 단순합니다. 작업 책상처럼 역할에 따라 나뉘어 있습니다.



data 폴더

실습에 사용할 제조 로그와 입력 데이터가 있습니다. (재료)



docs 폴더

알람 매뉴얼 같은 참고 문서가 있습니다. (설명서)



src 폴더

실행할 Python 코드가 있습니다. (실행 도구)



templates 폴더

프롬프트와 결과 보고서 양식이 있습니다. (빈 양식)



outputs 폴더

실행 결과가 저장됩니다. (완성된 결과물)



💡 핵심 정리: 프로젝트 폴더는 재료, 설명서, 실행 코드, 결과물을 나누어 둔 작업 책상입니다.

data, docs, src/day1 역할

data 폴더 — 재료

- sample_alarm_logs.csv: 교육용 알람 로그 파일
- sample_query.json: 사용자 요청, 설비 ID, 알람 코드 등 조건 정보

docs 폴더 — 설명서

- alarm_manual.md: 알람 의미와 확인 항목 제공

src/day1 폴더 — 실행 코드

- first_sample_run.py
- simple_chain_starter.py
- mini_graph_runner.py
- data와 docs의 재료를 처리하는 실습 코드



💡 핵심 정리: data와 docs는 재료이고, src/day1은 그 재료를 처리하는 실습 코드입니다.

templates, outputs 역할

📄 LLM에게 보낼 프롬프트와 결과 문서의 양식을 보관하는 곳입니다.

📁 templates 폴더 — 빈 양식

- first_sample_run_prompt.mustache: 첫 실행 실습용 프롬프트 양식
- simple_chain_prompt.mustache: Chain 실습용 프롬프트 양식
- 필요한 값을 채워 넣을 수 있는 빈 보고서 양식입니다.

📁 outputs 폴더 — 완성된 결과물

- 실행 결과가 저장되는 곳입니다.
- 실행 후 outputs/day1에 Markdown 결과 파일이 만들어집니다.
- templates = 빈 양식지 / outputs = 작성이 끝난 결과물

✅ 💡 핵심 정리: templates는 빈 양식이고 outputs는 작성이 끝난 결과물입니다.

6. [교육용 가상 시나리오] 제조 알람 로그와 기술 문서 조회 실습

[교육용 가상 시나리오] 제조 알람 로그와 기술 문서 첫 조회 실습

- 교육용 가상 알람 로그(sample_alarm_logs.csv)와 기술 문서(alarm_manual.md)를 처음 열어보는 실습입니다.
- 설비 ID, 알람 코드, 발생 건수 등 핵심 정보를 직접 확인합니다.
- AI Agent가 답변하기 전에 어떤 재료(데이터)를 수집하는지 흐름을 이해합니다.

제조 Log와 기술 문서의 역할

제조 Log란?

설비에서 발생한 일을 시간 순서대로 기록한 데이터입니다.

실습 파일: `sample_alarm_logs.csv`

포함 정보:

- 알람 발생 시각
- 라인, 공정, 설비 ID
- 알람 코드, 심각도
- 측정값, 반복 횟수
- 작업자 메모

기술 문서란?

알람의 의미와 확인 항목을 설명하는 자료입니다.

실습 파일: `docs/alarm_manual.md`

비유로 이해하기

제조 Log

진료 기록

기술 문서

진료 가이드



핵심 정리: 제조 Log는 진료 기록이고 기술 문서는 진료 가이드와 비슷합니다.

first_sample_run.py와 simple_chain_starter.py의 차이

1일차 실습에서는 먼저 데이터를 확인하고, 그다음 Chain 흐름을 확인합니다. 두 파일은 비슷해 보이지만 학습 목적이 다릅니다.

파일	핵심 목적	수강생이 봐야 할 것
first_sample_run.py	데이터를 처음 읽고 재료를 확인하는 실습	로그와 매뉴얼이 어떻게 읽히는지 확인
simple_chain_starter.py	정해진 순서의 Chain 흐름을 확인하는 실습	질문 → Prompt → LLM 응답 흐름 확인

두 파일 모두 교육용 제조 알람 데이터를 사용하지만, first_sample_run.py는 “재료 확인”에 가깝고, simple_chain_starter.py는 “업무 순서 실행”에 가깝습니다.

실습 흐름

1

first_sample_run.py

- sample_query.json 확인
- sample_alarm_logs.csv 확인
- alarm_manual.md 확인

2

simple_chain_starter.py

- 확인한 재료를 Prompt로 구성
- LLM 응답까지 연결



💡 핵심 정리: first_sample_run.py는 재료 확인 실습이고, simple_chain_starter.py는 Chain 실행 흐름을 확인하는 실습입니다.

first_sample_run.py 파일 역할

src/day1/first_sample_run.py는 1일차 첫 실행 실습 파일입니다. Agent가 답변하기 전에 사용할 재료를 처음 확인하는 실습입니다.

1

sample_query.json에서 사용자 요청 조건을 읽습니다.

2

sample_alarm_logs.csv에서 설비 ID와 알람 코드가 맞는 로그를 찾습니다.

3

alarm_manual.md 내용을 참고하여 프롬프트에 넣을 자료를 준비합니다.

4

llm_client.py의 generate_response(prompt)를 통해 실제 LLM 호출 흐름으로 연결합니다.

이 파일이 하는 일



핵심 정리: 이 파일은 Agent가 답변하기 전에 사용할 재료를 처음 확인하는 실습입니다.

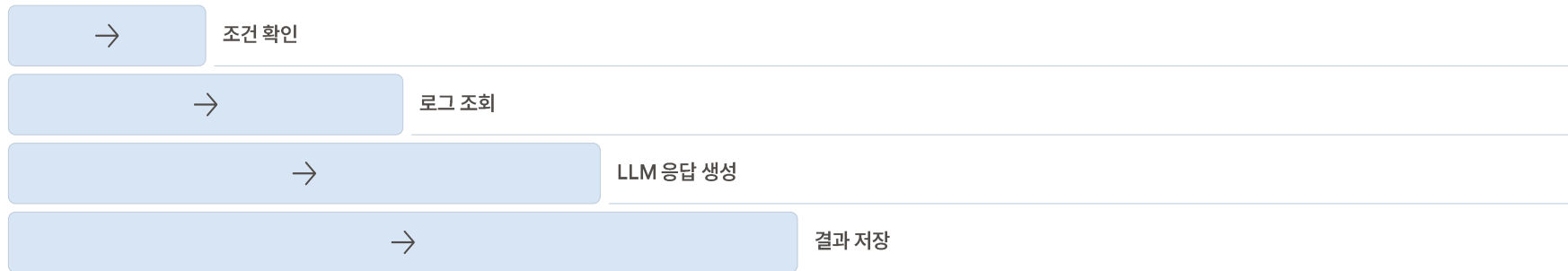


실행 명령

```
uv run python src/day1/first_sample_run.py
```

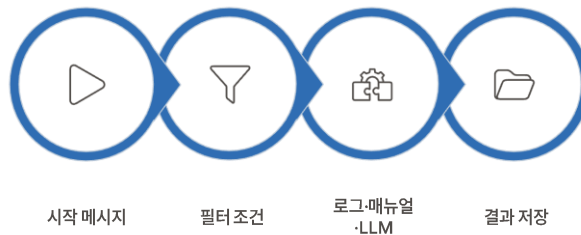
예상 출력 예시

first_sample_run.py를 실행하면 다음과 같은 순서로 출력이 나타납니다.



필터 조건 예시

항목	값
equipment_id	EQP-EV-03
alarm_code	ALM-TEMP-402



💡 핵심 정리: 예상 출력은 조건 확인 → 로그 조회 → LLM 응답 → 결과 저장 흐름을 보여줍니다.

실행 명령

```
uv run python src/day1/first_sample_run.py
```

출력 결과 해석

실행 결과를 볼 때는 LLM 응답만 보면 안 됩니다. 근거를 먼저 확인하는 것이 핵심입니다.

1

설비 ID와 알람 코드 확인

어떤 조건으로 필터링했는지 먼저 확인합니다.

2

관련 로그 수 확인

관련 로그가 많으면 반복 발생 가능성을 생각할 수 있습니다.

3

심각도와 작업자 메모 확인

WARNING과 CRITICAL이 함께 나타난다면 더 주의 깊게 볼 수 있습니다.

4

매뉴얼 내용 포함 여부 확인

어떤 매뉴얼 내용이 프롬프트에 포함되었는지도 확인해야 합니다.



핵심 정리: 결과 해석의 핵심은 응답보다 근거를 먼저 보는 것입니다.

설비 ID, 알람 코드, 발생 건수 정리

실습 후에는 결과를 직접 정리해야 합니다. 이 정리는 다음 Prompt 실습에 사용됩니다.

정리 예시

항목	내용
설비 ID	EQP-EV-03
알람 코드	ALM-TEMP-402
관련 라인	LINE-07 계열 교육용 라인
발생 건수	실행 결과에서 확인
주요 심각도	실행 결과에서 확인
측정값	실행 결과에서 확인
작업자 메모	실행 결과에서 확인
참고 문서	alarm_manual.md

왜 정리가 중요한가요?

회의 전에 자료를 정리해 두면 보고서 작성이 쉬워지는 것처럼, LLM에게 줄 정보도 먼저 정리해야 합니다.

  좋은 Prompt는 정리된 입력 자료에서 시작됩니다.



 핵심 정리: 좋은 Prompt는 정리된 입력 자료에서 시작됩니다.

다음 Prompt 실습 연결

✅ 💡 핵심 정리: 지금까지는 재료를 확인한 단계이고, 다음은 요리사에게 정확한 주문서를 작성하는 단계입니다.

지금까지 한 것

알람 로그와 기술 문서를 처음 확인했습니다. 데이터를 읽었다고 해서 LLM이 자동으로 좋은 답변을 만드는 것은 아닙니다.

❗ 읽은 정보를 LLM이 이해할 수 있는 작업 지시문 (Prompt)으로 정리해야 합니다.

다음 장에서 배울 것

- LLM에게 어떤 역할을 줄지
- 어떤 입력 데이터를 넣을지
- 어떤 출력 형식을 요구할지
- 어떤 표현을 금지할지

7. Prompt 기본 이해와 제조 장애 대응 프롬프트 작성

Prompt 기본 이해와 제조 장애 대응 프롬프트 작성

- Prompt란 단순한 질문이 아닌, 역할·상황·데이터·출력 형식을 포함한 작업 지시문입니다.
- 나쁜 프롬프트와 좋은 프롬프트를 비교하며 차이를 이해합니다.
- 제조 장애 대응 시나리오를 바탕으로 실제 프롬프트를 직접 작성해 봅니다.

Prompt란 무엇인가

단순한 질문이 아니라, 역할, 상황, 입력 데이터, 출력 형식, 주의사항을 함께 전달하는 요청서입니다.

Prompt = LLM에게 주는 작업 지시문

❏ "알람 원인 알려줘"

❌ 질문에 가까운 예시

"알람 원인 알려줘"

✅ 작업 지시문에 가까운 예시

"설비 EQP-EV-03에서 ALM-TEMP-402 알람이 반복 발생했습니다. 제공된 로그와 매뉴얼 근거를 바탕으로 원인 후보와 1차 확인 항목을 정리해 주세요."

Prompt에 포함되는 것들

역할

LLM이 어떤 역할을 해야 하는지



상황

어떤 상황인지 설명

입력 데이터

참고할 로그와 문서



출력 형식

어떤 형태로 답해야 하는지

주의사항

하면 안 되는 것들

✅ 💡 핵심 정리: Prompt는 LLM이 어떤 방식으로 답해야 하는지 알려주는 주문서입니다.

나쁜 프롬프트 예시



🔔 나쁜 프롬프트는 주소가 없는 택배처럼 원하는 결과에 도착하기 어렵습니다.

나쁜 프롬프트란?

너무 짧고 정보가 부족한 지시문입니다.

| ❌ "알람 원인 알려줘"

이렇게만 쓰면 LLM은 다음을 알 수 없습니다.

어떤 설비인지

어떤 알람인지

어떤 로그를 참고해야 하는지

왜 위험한가요?

그 결과 LLM은 일반적인 추측을 할 가능성이 높습니다.

제조 업무에서는 이런 추측이 위험할 수 있습니다. 실제 확인되지 않은 원인을 단정하면 잘못된 판단으로 이어질 수 있기 때문입니다.

비유로 이해하기

나쁜 프롬프트는 주소가 없는 택배처럼 원하는 결과에 도착하기 어렵습니다.

좋은 프롬프트 예시

좋은 프롬프트는 역할, 상황, 근거, 출력 형식, 주의사항을 포함합니다.

✅ "설비 EQP-EV-03에서 ALM-TEMP-402 알람이 반복 발생했습니다. 제공된 로그와 매뉴얼 근거를 바탕으로 원인 후보와 1차 확인 항목을 정리해 주세요. 확인되지 않은 원인은 단정하지 말고 가능성으로 표현해 주세요."

상황 포함

EQP-EV-03, ALM-TEMP-402 알람 반복 발생

근거 포함

제공된 로그와 매뉴얼 근거를 바탕으로

출력 형식 포함

원인 후보와 1차 확인 항목 정리

제약 조건 포함

단정하지 말고 가능성으로 표현



핵심 정리: 좋은 프롬프트는 LLM이 안전하고 구조적으로 답하도록 안내하는 작업 요청서입니다.

좋은 프롬프트 4요소

1

역할

LLM에게 제조 장애 대응 보조 Agent 역할을 줍니다.

2

입력 데이터

설비 ID, 알람 코드, 로그 요약, 매뉴얼 근거가 여기에 해당합니다.

3

출력 형식

예를 들어 Markdown 리포트 형식으로 답하게 할 수 있습니다.

4

제약 조건

원인을 단정하지 말고, 근거 없는 내용을 추측하지 말라는 조건입니다.



핵심 정리: 좋은 프롬프트는 역할, 데이터, 형식, 주의사항을 함께 담아야 합니다.

역할과 입력 데이터

Prompt 템플릿 사용

실습 예제에서는 아래 파일을 사용합니다.

templates/day1/simple_chain_prompt.mustache

템플릿에 들어가는 입력 정보

- 사용자 요청
- line_id
- process_name
- equipment_id
- alarm_code
- 참고 매뉴얼 내용

어떻게 동작하나요?

Python 코드는 data와 docs에서 필요한 값을 읽고, 템플릿의 빈칸에 채워 넣습니다.

데이터	템플릿
요리 재료	레시피 양식
Prompt	
완성된 주문서	

✔ 💡 핵심 정리: Prompt는 입력 데이터를 LLM이 이해할 수 있는 문장으로 정리한 것입니다.

출력 형식과 제약 조건

출력 형식 — 보고서 양식

LLM에게는 어떤 형식으로 답해야 하는지도 알려주어야 합니다. 이번 실습에서는 **Markdown** 리포트 형식을 사용할 수 있습니다.

요청 요약

알람 의미

원인 후보

1차 확인 항목

권장 조치 방향

추가 확인 필요 사항

제약 조건 — 안전벨트

제약 조건도 중요합니다.



▲ 실제 조치처럼 단정하지 말 것



▲ 로그와 매뉴얼에 없는 내용을 추측하지 말 것

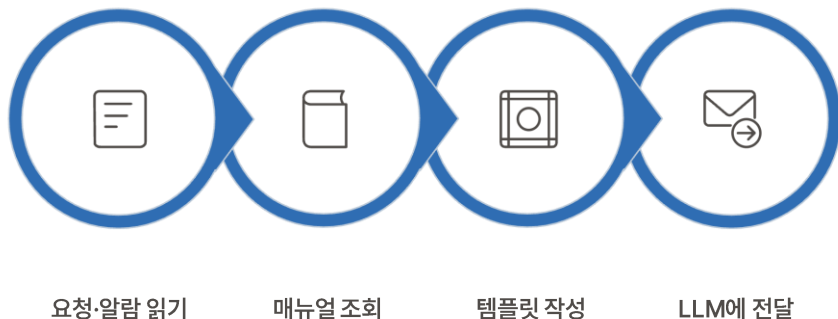
출력 형식은 보고서 양식이고 제약 조건은 안전벨트입니다.



핵심 정리: 출력 형식은 보고서 양식이고 제약 조건은 안전벨트입니다.

제조 장애 대응 프롬프트 작성 실습

이번 실습에서는 제조 장애 대응 프롬프트가 어떻게 만들어지는지 확인합니다.



수강생이 확인할 것

템플릿에 어떤 정보가 들어가는지 확인하면 됩니다. 코드 전체를 외울 필요는 없습니다.

실습의 목표

좋은 답변이 좋은 작업 지시에서 시작된다는 점을 이해하는 것입니다.



핵심 정리: 프롬프트 작성 실습의 목표는 좋은 답변이 좋은 작업 지시에서 시작된다는 점을 이해하는 것입니다.

실제 LLM 응답 비교

LLM 응답의 특성

실제 LLM을 사용할 때는 같은 프롬프트라도 응답 문장이 조금씩 달라질 수 있습니다.



글자가 완전히 같은지보다 필요한 구조가 포함되었는지를 확인해야 합니다.

좋은 응답 체크리스트

- 원인 후보가 가능성으로 표현되었는지
- 매뉴얼 근거가 반영되었는지
- 추가 확인 필요 사항이 분리되었는지

✖ 나쁜 응답

말이 화려하지만 근거가 없는 응답

✔ 좋은 응답

근거와 주의사항이 분명한 응답



💡 핵심 정리: 실제 LLM 응답은 표현보다 구조와 근거를 중심으로 평가해야 합니다.

Prompt 실습 정리

Prompt란?

LLM에게 주는 작업 지시문입니다.

좋은 Prompt 4요소

역할 + 입력 데이터 + 출력 형식 + 제약 조건

핵심 제약 조건

"근거 없는 내용은 추측하지 말 것", "원인을 단정하지 말 것"

다음 단계

이 Prompt가 실제 실행 흐름(Chain) 속에서 어떻게 사용되는지 확인합니다.



💡 핵심 정리: Prompt는 Chain 실행의 출발점입니다.

8장. Chain 기반 업무 흐름 실습

Chain 기반 업무 흐름 실습

- Chain이란 여러 작업을 정해진 순서대로 코드로 연결한 실행 흐름입니다.
- 사용자 질문 → 로그/문서 조회 → Prompt 생성 → LLM 호출 → 결과 저장의 직선형 흐름을 실습합니다.
- `simple_chain_starter.py`를 실행하며 실제 LLM 호출 결과를 확인하고 해석합니다.
- Chain의 장점과 한계를 이해하고, 다음 Graph 실습으로 연결합니다.

Chain이란 무엇인가

Chain = 업무 순서를 코드로 연결한 구조

여러 작업을 정해진 순서대로 연결한 실행 흐름입니다.

제조 장애 대응 예제로 보면

- 사용자 질문 읽기
- Prompt 만들기
- 실제 LLM API 호출
- LLM 응답 받아 결과 출력

Chain은 어떤 작업에 적합한가요?

복잡한 판단보다 순서가 중요한 작업에 적합합니다.

비유로 이해하기 — 요리 순서

제공

손님에게 서빙

담기

그릇에 담기

조리

요리하기

재료 준비

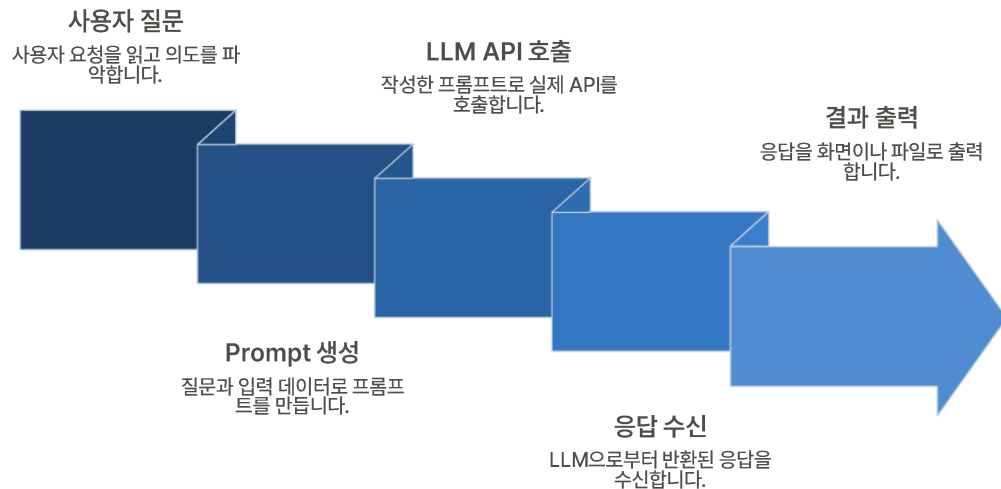
재료 손질



핵심 정리: Chain은 업무 순서를 코드로 연결한 구조입니다.

Chain 실행 흐름

1일차 Chain 실습의 흐름은 단순합니다. 한 단계가 끝나면 다음 단계로 넘어가는 직선형 업무 흐름입니다.



💡 핵심 정리: Chain은 한 단계가 끝나면 다음 단계로 넘어가는 직선형 업무 흐름입니다.

simple_chain_starter.py 파일 역할

파일 위치 src/day1/simple_chain_starter.py Chain 기반 업무 흐름을 보여주는 1일차 핵심 실습 파일입니다.	이 파일이 하는 일 사용자 요청을 읽습니다.	이 파일이 하는 일 알람 매뉴얼 내용을 참고하여 Prompt를 만듭니다.
이 파일이 하는 일 llm_client.py의 generate_response(prompt)를 통해 LLM 응답을 받습니다.		

이 파일의 목적

특정 LLM 제품의 복잡한 사용법보다, 아래 흐름을 이해하는 데 목적이 있습니다.

  질문이 어떻게 Prompt가 되고, Prompt가 어떻게 LLM 응답으로 이어지는가?

비유로 이해하기

이 파일은 Chain의 흐름을 눈으로 확인하기 위한 실습 파일입니다.

  핵심 정리: 이 파일은 **Chain**의 흐름을 눈으로 확인하기 위한 실습 파일입니다.

실행 명령

```
uv run python src/day1/simple_chain_starter.py
```

실제 LLM 호출 결과 확인

실행 흐름

simple_chain_starter.py를 실행하면 사용자 요청과 알람 매뉴얼 내용이 Prompt에 반영됩니다. 이후 llm_client.py를 통해 실제 LLM 호출 흐름으로 응답을 생성합니다.

Cloud LLM

인터넷을 통해 외부 LLM 서비스를 호출합니다.

Local LLM

로컬 환경에서 LLM을 실행합니다.

수업 환경에 따라 호출 방식은 달라질 수 있습니다.

✔️ 💡 핵심 정리: 실제 LLM 호출 결과는 Prompt 품질을 확인하는 중요한 자료입니다.

중요한 보안 원칙

⚠️ ▲ API Key나 비밀번호는 화면에 출력하지 않습니다.

결과 파일에서 확인할 수 있는 것

- 사용된 Prompt
- LLM 응답

실행 명령

```
uv run python src/day1/simple_chain_starter.py
```

실행 결과 해석

실행 결과를 해석할 때는 응답 문장만 보지 않습니다. 완성된 요리만 보는 것이 아니라 재료와 조리 순서도 함께 확인하는 것과 같습니다.

1

사용자 질문 확인

사용자 질문이 무엇이었는지 먼저 확인합니다.

2

Prompt 내용 확인

어떤 설비 ID, 알람 코드, 매뉴얼 근거가 들어갔는지 확인합니다.

3

응답 표현 방식 확인

LLM 응답이 원인을 단정하지 않고 가능성으로 표현했는지 봅니다.

4

추가 확인 사항 포함 여부

추가 확인 필요 사항이 포함되었는지도 확인합니다.



핵심 정리: Chain 결과는 입력, Prompt, 응답을 함께 봐야 이해할 수 있습니다.

Chain의 장점과 한계

✓ Chain의 장점

- 단순하고 이해하기 쉽습니다.
- 정해진 순서로 실행되어 초보자가 흐름을 따라가기 좋습니다.
- 사용자 질문 → Prompt → LLM 호출 → 응답 구조가 명확합니다.

⚠ Chain의 한계

Chain은 항상 같은 순서로 실행된다는 한계가 있습니다.



예를 들어 사용자 질문에 알람 코드가 없다면, 바로 LLM을 호출하기보다 추가 질문이 필요할 수 있습니다.

비유로 이해하기

Chain은 직선 도로와 같습니다. 목적지가 정해져 있을 때는 빠르지만, 상황에 따라 다른 길이 필요할 때는 한계가 있습니다.



핵심 정리: Chain은 직선형 업무에는 좋지만 조건에 따라 달라지는 업무에는 한계가 있습니다.

Graph 실습 연결 질문

Chain의 한계 상황

지금까지는 정해진 순서대로 실행되는 Chain을 확인했습니다. 하지만 실제 제조 장에 대응에서는 상황에 따라 흐름이 달라질 수 있습니다.

✖ 정보 부족

"알람 원인 알려줘"

→ 설비 ID와 알람 코드가 부족

→ 추가 정보 요청 필요

✔ 정보 충분

EQP-EV-03 + ALM-TEMP-402 포함

→ 로그 조회와 매뉴얼 확인으로 진행

다음 장에서 배울 것 — Graph

이런 조건부 흐름을 Graph로 살펴봅니다.

Graph란?

상황에 따라 길이 갈라지는 내비게이션처럼, Agent가 조건에 따라 다른 경로를 선택하는 실행 구조입니다.



✖ Chain = 직선 도로

Graph = 상황에 따라 경로를 바꾸는 내비게이션



💡 핵심 정리: Graph는 상황에 따라 길이 갈라지는 Agent 실행 구조입니다.